

AIR QUALITY MONITORING AND ALERT SYSTEM

CONSOLIDATED TECHNICAL DESIGN REPORT

ER Model • Relational Schema • Normalization • Database Automation • Application Flow

CMPE 232 - Database Systems
TED University • Spring 2026

Air Quality Analytics Team (Sec02-Group16)

Beyza Yağmur Er • Semih Can Kasar • Güner Tugay Arısoy

Public repository edition

Student identification numbers and private credentials are intentionally excluded.

Executive Summary

The Air Quality Monitoring and Alert System is a relational database-backed web application for organizing, analyzing, and presenting global air-quality information. The system stores countries, cities, pollutant-specific AQI measurements, AQI categories, and automatically generated alerts. Its database design begins from a conceptual ER model, is transformed into a normalized relational schema, and uses constraints and SQL triggers to maintain consistency and automate derived behavior.

This consolidated report combines the project's ER-model documentation and relational/database-structure documentation into a single public-facing technical reference. It preserves the original academic design while removing student identification numbers and avoiding repository-specific instructions.

Technical focus

Relational modeling, 3NF normalization, staging-table import, foreign-key integrity, update-tracking triggers, automatic Overall AQI recalculation, and AQI-based alert generation.

Document Scope

- Dataset structure and ingestion strategy.
- Conceptual ER model and entity responsibilities.
- Relational schema, keys, constraints, and cardinalities.
- Normalization rationale from raw import data to 3NF tables.
- Database triggers and automated AQI/alert behavior.
- Final application flow, user/admin capabilities, and implementation architecture.

Contents

1	Project Context and Objectives
2	Dataset and Raw Data Structure
3	Conceptual ER Model
4	Relational Schema and Integrity Constraints
5	Normalization Strategy
6	Database Automation and Trigger Logic
7	Data Import and Transformation Pipeline
8	Web Application Architecture and Roles
9	User Interface Flow
10	Technical Limitations and Extension Points
11	Team and References

1. Project Context and Objectives

The project was designed to manage air-quality information across countries and cities in a structured relational database. The core measurements are Air Quality Index (AQI) values associated with pollutants such as carbon monoxide (CO), ozone (O3), nitrogen dioxide (NO2), and particulate matter (PM2.5).

Beyond data storage, the system classifies AQI values, supports location-based comparison, records pollutant measurements, and generates alerts when air quality reaches unsafe categories. The web interface exposes the database through role-aware pages for browsing, filtering, analysis, and administrative maintenance.

Primary objectives

- Represent countries, cities, pollutants, AQI categories, measurements, and alerts without unnecessary duplication.
- Maintain referential integrity between geographic, pollutant, category, measurement, and alert data.
- Transform the original wide CSV structure into normalized relational tables.
- Automate repeated database behavior using SQL triggers.
- Provide a usable web interface for both read-oriented users and administrative users.

High-level application view

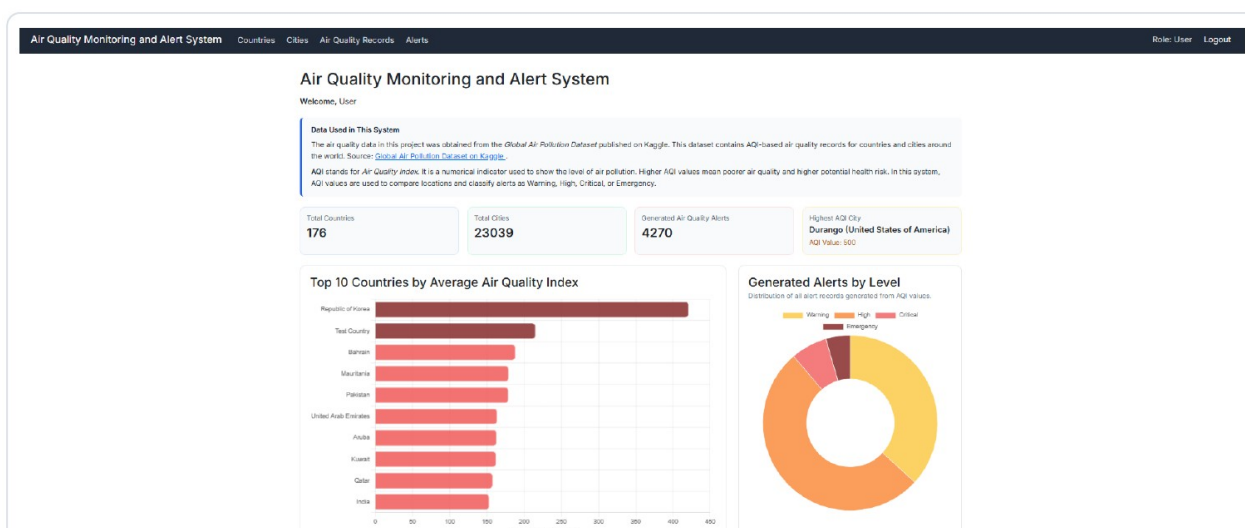


Figure 1. Final user dashboard showing dataset coverage, AQI comparison, and generated-alert distribution.

2. Dataset and Raw Data Structure

The project uses the Global Air Pollution Dataset distributed through Kaggle. The source CSV contains one row per city-level observation and combines overall AQI information with pollutant-specific AQI values in a wide format. This makes the file convenient for import, but it also contains repeated country/city text and multiple pollutant column groups that motivate normalization.

File	global air pollution dataset.csv
Raw records	23,464 rows
Columns	12
Distinct country values	176
Overall AQI range	6 - 500
Missing cells in raw CSV	428
Source	Kaggle - Global Air Pollution Dataset (Hasib Al Muzdadid)

Raw columns

Column	Purpose
Country	Geographic identifier
City	Geographic identifier
AQI Value	AQI numeric value
AQI Category	AQI classification label
CO AQI Value	AQI numeric value
CO AQI Category	AQI classification label
Ozone AQI Value	AQI numeric value
Ozone AQI Category	AQI classification label
NO2 AQI Value	AQI numeric value
NO2 AQI Category	AQI classification label
PM2.5 AQI Value	AQI numeric value
PM2.5 AQI Category	AQI classification label

Repository note

The dataset should be represented in the repository, but the safest public-release approach is to keep a data/README.md with the Kaggle source and expected filename unless redistribution permission is explicit. Kaggle currently labels this dataset license as “Other (specified in description).”

3. Conceptual ER Model

The conceptual model centers on AirQualityRecord. Each record represents the AQI value for a specific pollutant in a specific city and is classified by one AQICategory. Country and City provide the geographic hierarchy, Pollutant provides measurement type, and Alert/AlertNote capture unsafe conditions and supporting notes.

Pollutant is modeled as a superclass with GasPollutant and ParticulatePollutant subclasses. AlertNote is modeled as a weak entity whose identity depends on Alert and a partial key (NoteNo). RawAirQualityImport is a staging structure rather than a long-term normalized entity.

III. Final ER Schema

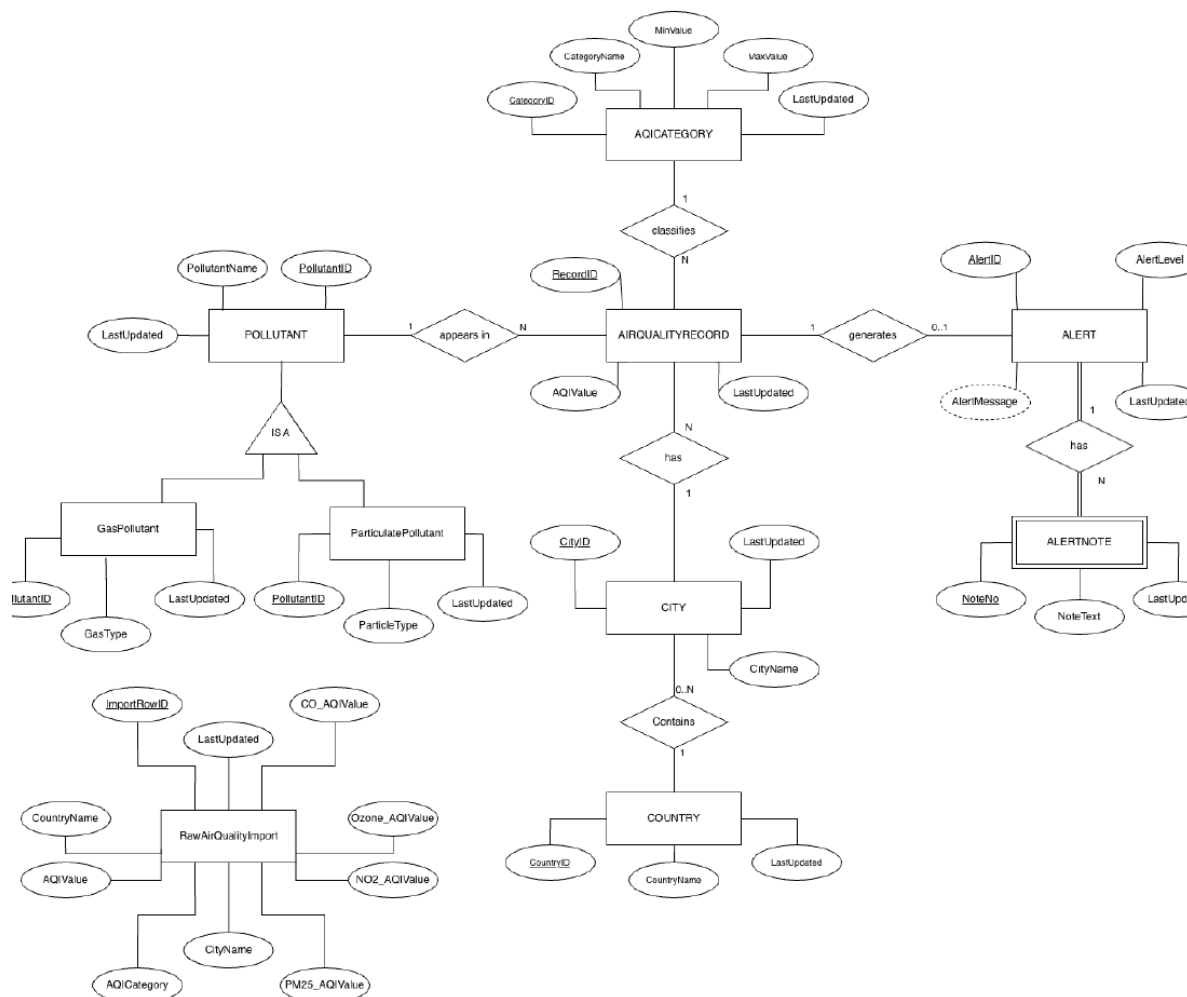


Figure 2. Final ER schema from the original project design documentation.

3.1 Entity Responsibilities

Entity	Responsibility
Country	Countries represented in the dataset; unique country name.
City	Cities associated with exactly one country.
Pollutant	Superclass for supported pollutant types.
GasPollutant	Gas subtype metadata (CO, Ozone, NO2).
ParticulatePollutant	Particulate subtype metadata (PM2.5).
AQICategory	Named AQI ranges such as Good, Moderate, Unhealthy, and Hazardous.
AirQualityRecord	Central measurement entity connecting city, pollutant, category, and AQI value.
Alert	At most one alert per air-quality record.
AlertNote	Weak entity for multiple notes attached to an alert.
RawAirQualityImport	Wide-format staging table used before transformation.

Conceptual relationship summary

Country contains Cities; Cities and Pollutants meet through AirQualityRecord; each record is classified by one AQICategory; a record may generate at most one Alert; and each Alert can own multiple AlertNotes.

4. Relational Schema and Integrity Constraints

The ER model is implemented through normalized SQL Server tables. Primary keys uniquely identify rows, foreign keys preserve parent-child relationships, unique constraints prevent duplicate logical entities, and CHECK constraints reject invalid AQI values.

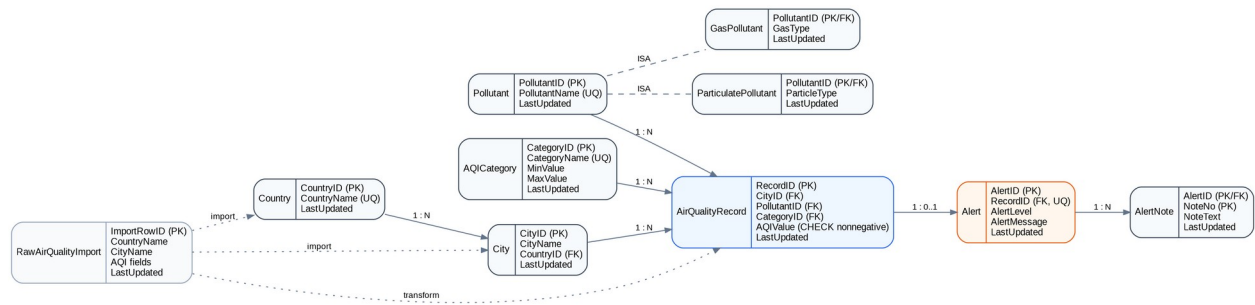


Figure 3. Consolidated relational schema overview.

Relationship	Cardinality	Implementation
Country → City	1 : N	City.CountryID → Country.CountryID
City → AirQualityRecord	1 : N	AirQualityRecord.CityID → City.CityID
Pollutant → AirQualityRecord	1 : N	AirQualityRecord.PollutantID → Pollutant.PollutantID
AQICategory → AirQualityRecord	1 : N	AirQualityRecord.CategoryID → AQICategory.CategoryID
AirQualityRecord → Alert	1 : 0..1	Alert.RecordID is FK and UNIQUE
Alert → AlertNote	1 : N	AlertNote.AlertID → Alert.AlertID
Pollutant → subtype tables	ISA	Subtype PK is also FK to Pollutant

Key integrity rules

- CountryName is unique and not null.
- City uses a composite uniqueness rule on (CityName, CountryID).
- PollutantName and CategoryName are unique.
- AirQualityRecord.AQIValue must be non-negative.
- Alert.RecordID is unique, enforcing at most one alert for a record.
- AlertNote uses the composite primary key (AlertID, NoteNo).

5. Normalization Strategy

The raw CSV is represented by RawAirQualityImport and contains repeated country names, city names, overall AQI fields, and separate pollutant-specific value/category columns. The normalized database decomposes this structure so that each concept is stored once and connected by keys.

Normal Form	Design Action	Result
1NF	Remove repeating pollutant groups from the long-term schema; represent measurements as rows in AirQualityRecord and pollutant types in Pollutant.	Atomic relational attributes and a consistent measurement structure.
2NF	Ensure non-key attributes depend on the whole primary key. AlertNote.NoteText depends on the composite key (AlertID, NoteNo); single-key tables depend on their PK.	No partial dependency in the normalized tables.
3NF	Separate AQI category metadata from measurements and country metadata from cities; keep pollutant classification in dedicated superclass/subclass tables.	No avoidable transitive dependency between non-key attributes.

Why the staging table remains

RawAirQualityImport is intentionally denormalized because it mirrors the external CSV and simplifies bulk import. It is a transient ingestion structure; the normalized tables are the authoritative application model.

Example decomposition

Raw CSV row

Country | City | AQI Value | CO AQI Value | Ozone AQI Value | NO2 AQI Value | PM2.5 AQI Value

Normalized representation

Country(country)

└ City(city)

└ AirQualityRecord(city, pollutant=CO, value=...)

└ AirQualityRecord(city, pollutant=Ozone, value=...)

└ AirQualityRecord(city, pollutant=NO2, value=...)

└ AirQualityRecord(city, pollutant=PM2.5, value=...)

└ AirQualityRecord(city, pollutant=Overall AQI, value=...)

6. Database Automation and Trigger Logic

SQL triggers are used for two categories of automation: modification tracking and domain-specific AQI/alert logic. The design documentation defines LastUpdated triggers for persistent entities and specialized triggers on AirQualityRecord.

6.1 LastUpdated tracking

After UPDATE operations, LastUpdated is set to SYSDATETIME() for the affected rows. Trigger nesting checks are used to avoid recursive update behavior.

```
AFTER UPDATE
SET NOCOUNT ON
IF TRIGGER_NESTLEVEL() > 1 RETURN
UPDATE target
SET LastUpdated = SYSDATETIME()
FROM target JOIN inserted ON primary_key
```

6.2 Overall AQI recalculation

When pollutant-specific AirQualityRecord rows change, the system identifies affected cities, calculates the maximum AQI among individual pollutants, maps that value to an AQICategory range, and MERGES the resulting Overall AQI record back into AirQualityRecord.

6.3 Alert generation

AQI Category	Alert Level
Good / Moderate	No generated alert
Unhealthy for Sensitive Groups	Warning
Unhealthy	High
Very Unhealthy	Critical
Hazardous	Emergency

The alert trigger evaluates changed Overall AQI records and uses MERGE semantics so an existing alert is updated while a new one is inserted when required. The one-alert-per-record rule is reinforced by the unique RecordID relationship.

7. Data Import and Transformation Pipeline

The import process separates external data ingestion from application data. The raw CSV is loaded into RawAirQualityImport, text values are trimmed and deduplicated, lookup entities are created, and measurements are transformed into AirQualityRecord rows. MERGE operations update existing city-pollutant records or insert new ones.



Figure 4. Dataset-to-application transformation and automation pipeline.

Import sequence

Step	Operation
1	Load the external CSV into RawAirQualityImport.
2	Insert distinct Country values that do not already exist.
3	Insert distinct City values using the resolved CountryID.
4	Resolve pollutant and AQI-category identifiers.
5	MERGE Overall AQI and pollutant-specific AQI values into AirQualityRecord.
6	Let recalculation/alert triggers keep derived records consistent after changes.

Idempotent import behavior

The documented MERGE and NOT EXISTS patterns reduce accidental duplication when import scripts are run more than once.

8. Web Application Architecture and Roles

The completed application is implemented as an ASP.NET MVC web application using C#, Entity Framework, LINQ, Razor Views, SQL Server, Bootstrap, and jQuery. Entity Framework maps the relational database into application models while controllers coordinate filtering, CRUD operations, and page navigation.

Layer	Technology	Responsibility
Presentation	Razor Views, HTML/CSS, Bootstrap, jQuery	Login, dashboard, tables, filters, CRUD forms
Application	ASP.NET MVC controllers and authorization filters	Request handling, role checks, query orchestration
Data access	Entity Framework + LINQ	Database queries and updates through mapped entities
Database	Microsoft SQL Server	Normalized tables, constraints, views, triggers, staging/import

Role behavior

Role	Capabilities
User	Read-oriented access to dashboard, Countries, Cities, Air Quality Records, and Alerts; search/filter/sort/pagination.
Admin	User capabilities plus creation, editing, deletion, and record-management actions exposed by administrative pages.

Separation of concerns

The web interface does not replace the database rules. Constraints and triggers remain responsible for core integrity and automation even when records are created or edited through the application.

9. User Interface Flow

The final interface organizes the system around a role-aware login and a common navigation bar. Both roles can reach the major data views; administrative actions are additionally exposed to Admin users.

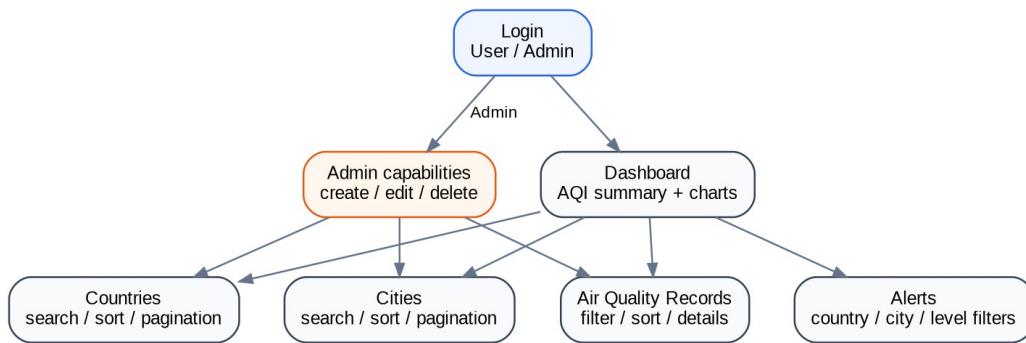


Figure 5. Consolidated final application navigation and role flow.

Primary pages

Page	Function
Login	Select User/Admin login type; Admin login includes credentials.
Dashboard	Dataset summary, highest AQI information, country AQI comparison chart, and alert-level distribution.
Countries	Country search, sorting, pagination; Admin create/edit/delete actions.
Cities	City search, sorting, pagination; Admin create/edit/delete actions.
Air Quality Records	Country/city/pollutant filters, sorting, pagination, detailed record management.
Alerts	Country/city/alert-level filtering and generated alert messages.

10. Technical Limitations and Extension Points

The current project is a database-centered monitoring and analysis application rather than a live sensor ingestion platform. Its strongest contribution is the relational model and the automation around AQI records and alerts. The following extension points naturally follow from the documented design.

- Replace periodic/static dataset import with scheduled API or sensor ingestion while preserving RawAirQualityImport as a staging layer.
- Add explicit audit/history tables if full change history is required beyond LastUpdated timestamps.
- Add database indexes based on production query plans for high-volume country/city/filter operations.
- Introduce stronger authentication/identity management if the project evolves beyond an academic deployment.
- Add automated integration tests for trigger behavior, AQI-category boundaries, and role-based CRUD operations.
- Add source/license metadata to the data-ingestion workflow so dataset provenance is always visible.

Repository packaging recommendation

For a public GitHub repository, keep technical documentation and raw data concerns separate. The project overview and this report belong under docs/. Dataset metadata belongs under data/.

```
air-quality-monitoring-alert-system/
├─ docs/
│  └─ Air_Quality_Monitoring_Alert_System_Project_Overview.pdf
│  └─ Air_Quality_Monitoring_Alert_System_Technical_Report.pdf
├─ data/
│  └─ README.md    # source, columns, expected filename, download link
├─ Controllers/
├─ Views/
├─ Content/
└─ ...
```

Raw CSV publication

Do not place the dataset inside docs/. If redistribution terms are verified, use data/global_air_pollution_dataset.csv. If terms are unclear, keep only data/README.md with attribution and a Kaggle download link.

11. Team and References

Academic context

Course	CMPE 232 - Database Systems
University	TED University
Semester	Spring 2026
Group	Air Quality Analytics Team (Sec02-Group16)

Team

Member	Documented contribution
Beyza Yağmur Er	SQL/database work; relational modeling; report preparation.
Semih Can Kasar	SQL/database work; relational modeling; report preparation.
Güner Tugay Arısoy	UI design; relational modeling.

References

- Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems, 7th ed., Pearson.
- Microsoft. Transact-SQL Reference (Database Engine).
- JGraph Ltd. draw.io.
- Kaggle. Global Air Pollution Dataset, Hasib Al Muzdadid. <https://www.kaggle.com/datasets/hasibalmuzdadid/global-air-pollution-dataset/data>
- World Health Organization. Air pollution and air-quality background resources.
- TED University CMPE 232 course materials and project design documentation.

Source documents consolidated in this edition

- Term Project Design Report for ER Model.
- Term Project Design Report for Relational Model and Database Structures.
- Completed project report and final UI-flow artifact.
- Final application screenshots and project overview report.

Public-release privacy

This consolidated version includes team names but intentionally omits student identification numbers and any machine-specific credentials.